

FUNCIONES



Funciones

Cuando creamos nuestros propios programas nos damos cuenta de que muchas de las tareas que implementamos se repiten o presentan de forma similar pero con algunos cambios.

Entonces aparece la necesidad de agrupar este código repetido o similar, a las agrupaciones de código se les denominan **funciones** las cuales se pueden ejecutar múltiples veces gracias a un nombre único que las identifica.



Funciones

Para comunicarse con nuestro proceso principal las funciones pueden recibir y devolver datos manipulados. Un ejemplo de una función que conocemos es `len()` que nos permite saber la cantidad de elementos de una colección.

*Recordemos que a esta función hay que pasarle el elemento del cual queremos saber la longitud y devuelve un valor entero con la longitud, a este valor se le denomina **valor de retorno**.*

```
>>> len("Hola")  
4
```

DEF



¿De qué se trata?

La sentencia **def** sirve para crear funciones definidas por el usuario.

Una definición de función es una sentencia ejecutable.



Sintaxis para una definición de función

```
>>> def NOMBRE(PARÁMETROS):  
        SENTENCIAS  
        RETURN [EXPRESIÓN]
```

- **NOMBRE:** Es el nombre de la función.
- **PARÁMETROS:** Como vimos en la clase 8 hay scripts con argumentos, en las funciones, cuando recibe argumentos se les denominan parámetros.



Sintaxis para una definición de función

- **SENTENCIAS:** Es el bloque de código.
- **RETURN:** Es una sentencia de Python, le indica a la función que devolver cuando llamemos a la función.
- **EXPRESIÓN:** Es lo que devuelve la sentencia **return**



Definir funciones básicas

```
>>> def saludar():  
    print("Estoy saludando  
desde la función")
```

La llamamos usando:

```
>>> saludar()
```



Definir funciones más avanzadas



```
>>> def saludar_con_nombre(nombre):  
        saludando = print("Hola {}! ¿Cómo  
estás?".format(nombre))  
        return saludando
```

Y la llamamos usando:

```
>>> saludar_con_nombre("Juan")
```



Recomendaciones

1. Utilizar minúsculas
2. Las palabras se separan con guiones bajos _
3. Utilizar nombres autoexplicativos
4. No usar nombres que no definan lo que hace la función (ejemplo letras simples o palabras sin sentido con lo que haga la función)

Variables y Funciones



```
>>> def test():  
    variable_test = 10  
    print(variable_test)  
  
>>> print(variable_test)  
NameError Traceback (most recent call last)  
<ipython-input-4-667d7c7a2c02> in  
<module>()  
----> 1 print(variable_test)
```

NameError: name 'variable_test' is not defined

Hay que tener en cuenta
que las variables creadas
en una función no existen
fuera de la misma.



Variables y Funciones

Sin embargo, hay que tener cuidado con las variables fuera de las funciones al usarlas en una función, ya que no puede llegar a funcionar como queremos:

```
>>> variable_test = 10
>>> def test():
    variable_test = 155
    print(variable_test)
>>> test()
```

- 👁 El print le da prioridad a la variable dentro de la función antes que a la de afuera.

RETORNANDO VALORES



Return



Las funciones pueden comunicarse con el exterior de las mismas, al proceso principal del programa usando la **sentencia return**. **La comunicación con el exterior se hace devolviendo valores.**

A continuación, un ejemplo de función usando return:

```
>>> def saludar_con_nombre(nombre):  
        saludando =  
print("Hola {}! ¿Cómo  
estás?".format(nombre))  
        return saludando
```

Nota: Por defecto, las funciones retorna el valor None.



Return



Sin embargo hay que tener en cuenta que la función **termina** al devolver un **valor**, es decir, lo que escribamos después **no** se ejecutará:

```
>>> def saludar_con_nombre(nombre):  
        saludando = print("Hola {}! ¿Cómo  
estás?".format(nombre))  
        return saludando  
        print("Hola mundo!")
```



Es similar a un break!



Return



```
>>> def numero():  
    return  
6  
>>> s = numero() + 5  
>>> s  
11  
>>> a = numero() * 2 +  
5  
>>> a  
17
```

Los valores o variables retornados van a seguir siendo de un tipo de valor, por lo que podremos trabajarlo con lo que ya hemos visto



Return



Si vemos el tipo de dato de número, nos indicará que es un int:

```
>>> def numero():  
    return 6  
>>> type(numero())  
<class 'int'>
```

Por lo que no
podremos
sumar int a str
aunque sea una
función

```
>>> a = numero() + "hola"  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: unsupported operand type(s) for +: 'int' and  
'str'
```



Return



Algo interesante que pasa si devolvemos una colección es que **podemos utilizarla directamente desde la función** y hacer uso de las funciones internas de las colecciones:

```
>>> def lista():  
    return  
    [1,2,3,4,5]  
>>> print(lista()[1:3])
```

Sin embargo, cada vez que hagamos un **print** a una función la estaremos llamando, por lo que **lo ideal es asignarlo a una variable y trabajarlo desde ahí:**

```
>>> variable = lista()  
>>> variable[1:4]
```



Return multiple



Una característica interesante,
es la posibilidad de devolver
valores múltiples separados por
comas:

```
>>> def test():  
        return "Python", 20, [1,2,3]  
  
>>> test()  
  
('Python', 20, [1, 2, 3])
```

ENVIANDO VALORES



Enviando Valores a una función

Vimos como devolver valores y así comunicar una función con el exterior,
ahora **enviar información desde el exterior a la función.**

Para entender los conceptos más fácilmente vamos a trabajar alrededor de
un caso de estudio típico: **Crear una función que sume dos números y
retorne uno en su resultado.**



Enviando Valores a una función



Lo primero será definir una función la cual denominaremos como **suma** y recibirá 2 números con dos nombres como si fueran dos variables **numero1** y **numero2**, luego **retornamos la suma** entre ambos números.

```
>>> def suma(numero1, numero2):  
        return numero1 +  
numero2
```



Enviando Valores a una función



Lo que hacemos para indicar que se reciben valores es **crear dos variables separadas por una coma**. Cuando nosotros **llamemos a la función**, automáticamente, **se le asignarán a estas variables los números que enviemos**, siguiendo el mismo orden:

```
>>> r = suma(7, 5)
```

En este caso 7 será la variable **numero1** y 5
será la variable **numero2**



|

En este caso 5 será la variable **numero1** y 7 será la variable **numero2**. Hay que tener cuidado por como se pasan estos valores a la función, ya que si fuera otra operación matemática podría dar resultados muy distintos, como en una división o potencia.

|

Momentos de una función



Tenemos la **definición**

```
>>> def suma(numero1, numero2):  
    return numero1 +  
numero2
```

Y la **llamada**

```
>>> r = suma(7, 5)
```



¿Por qué es importante
diferenciar?

Momentos de una función



Durante la definición de la función, las variables o valores se denominan **parámetros**:

```
>>> def suma(numero1, numero2):  
    return numero1 +  
numero2
```

Y durante la llamada se le denominan **argumentos**, como los argumentos de los scripts.

```
>>> r = suma(7, 5)
```